

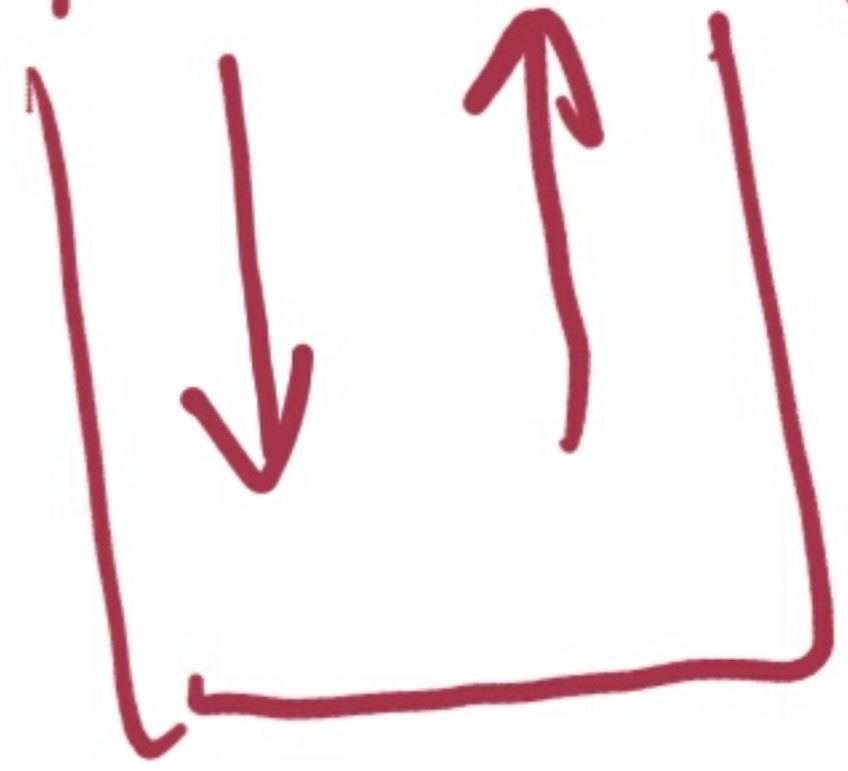
Dec 3

Stack & queue

Stack

A list with the restriction that insertion and deletion can be performed only from one end called top of the stack

Push Pop (Insertion and deletion have a reversed direction)



ex. Stack of books

Stack of plates $\Rightarrow$

LIFO

FIFO

Stack operations

① Push(x)

② Pop()

③ Top()

④ isEmpty()

⑤ isFull()

Tracing Example

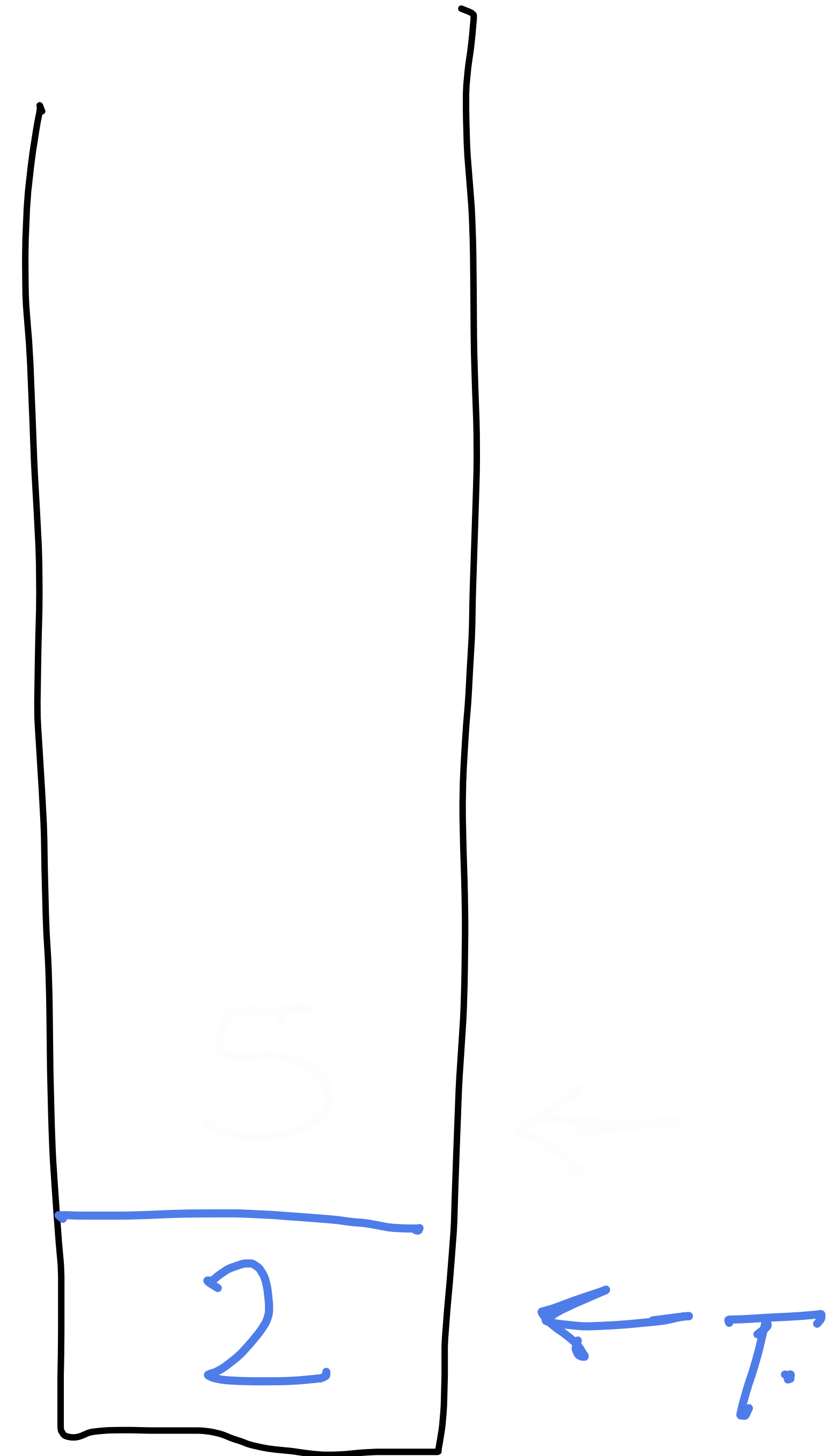
~~Push(2)~~

~~Push(5)~~

~~Top() → 5~~

~~isEmpty() → False~~

pop() → 5



Stack applications

- 1 Function Calls
- 2 ReCursion e.g.
- 3 Implementation of undo operation
- 4 expression parsing (check balanced parentheses)
- 5 Traverse any list by reverse order

Implementing Stack

```
graph TD; A[Implementing Stack] --- B[Using array]; A --- C[Using Linked List]
```

Using array

Using Linked List

I

Array-Based Stack

```
int Max_size = 10  
int Stack[Max_size]  
int Top = -1
```

push(x)

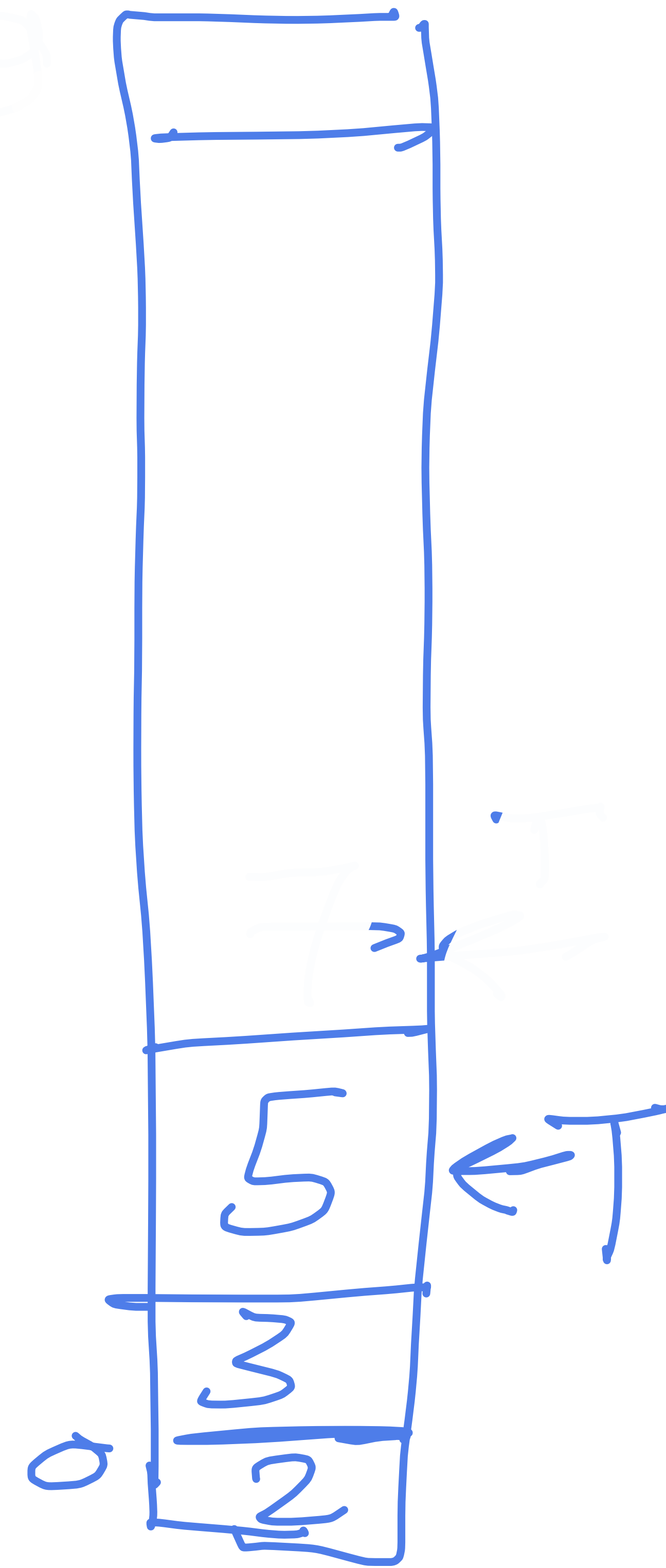
```
{ if (top == Max_size - 1) Print "error"  
  else { top++  
        Stack[top] = x  
      }  
}
```



Top = -1


```
POP ( )  
{ If (Top = -1) Print "error"  
  else { top -- ;  
        }  
}
```

Push and Pop are $O(1)$

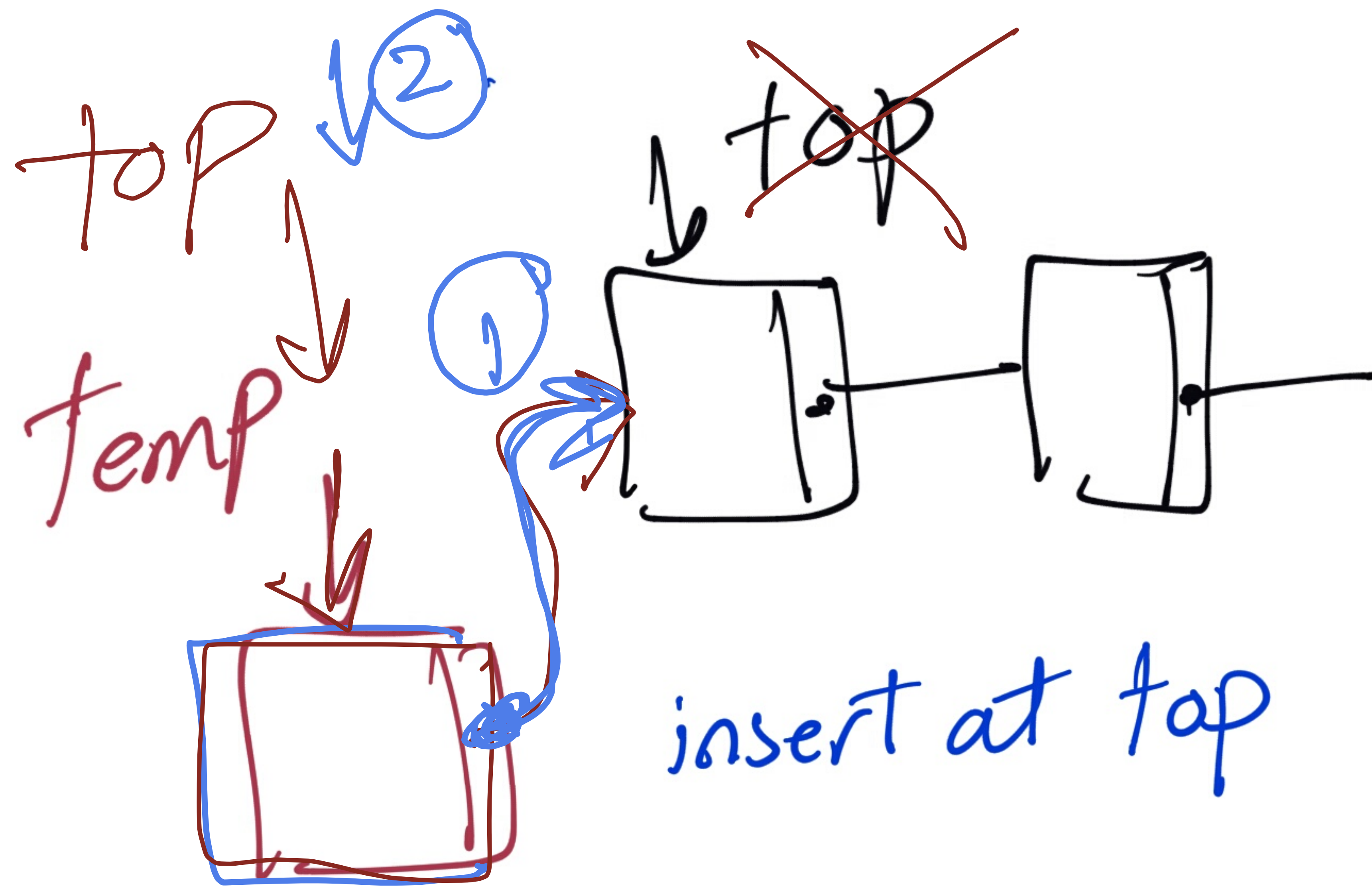


[2] Linked list - based stack

Initially, $Top = Null$

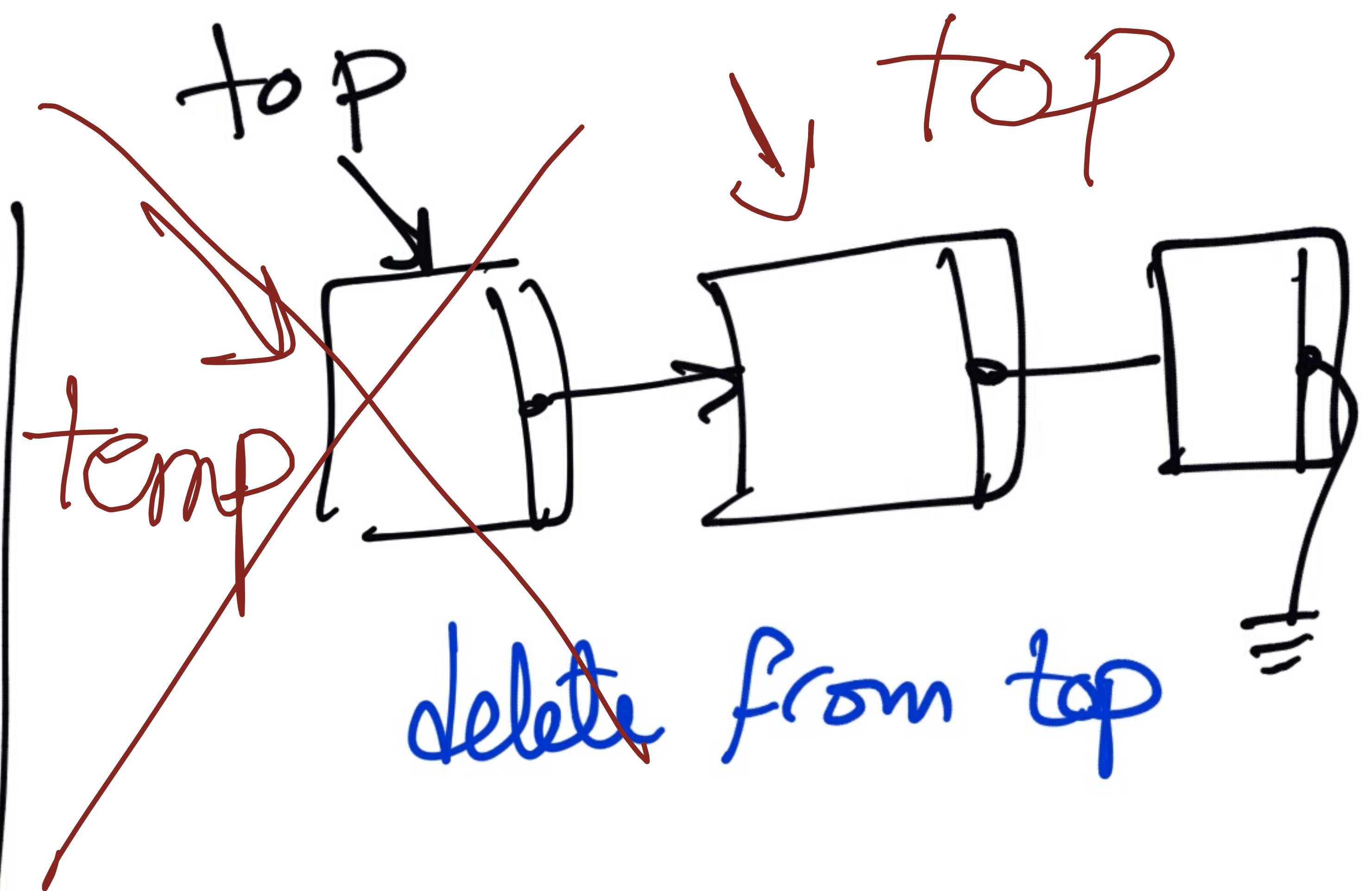
Push at top of stack $O(1)$

Pop at top of stack $O(1)$



insert at top

Push



Pop

Push (X)

{ // create node and fill it

temp = new node()

temp.data = X

temp → next = top

top = temp

}

pop ()
{ if (top = ^{empty} Null) $\Rightarrow$ Error

else { temp = top
top = temp $\rightarrow$ next
free (temp)
}

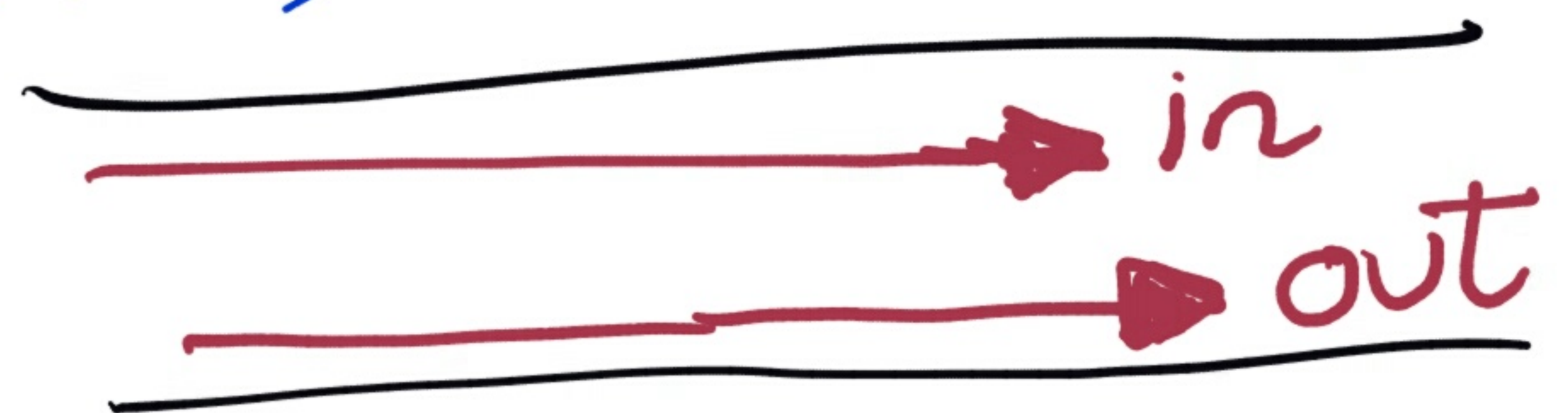
}

Queue (Buffer)

Queue

A list with the restriction that insertion can be performed at one end and deletion can be performed at the other end
add from ^{tail} rear, delete from front

FIFO



Insertion and deletion have the same direction

Queue Operations

- [1] enqueue at tail/rear
- [2] Dequeue from front
- [3] front ()
- [4] isEmpty
- [5] isfull

Queue Applications

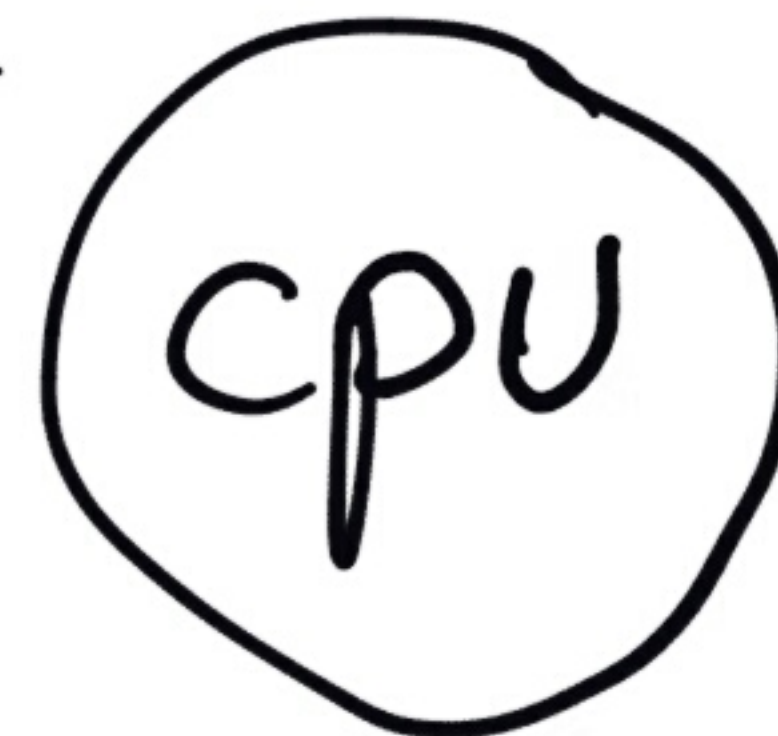
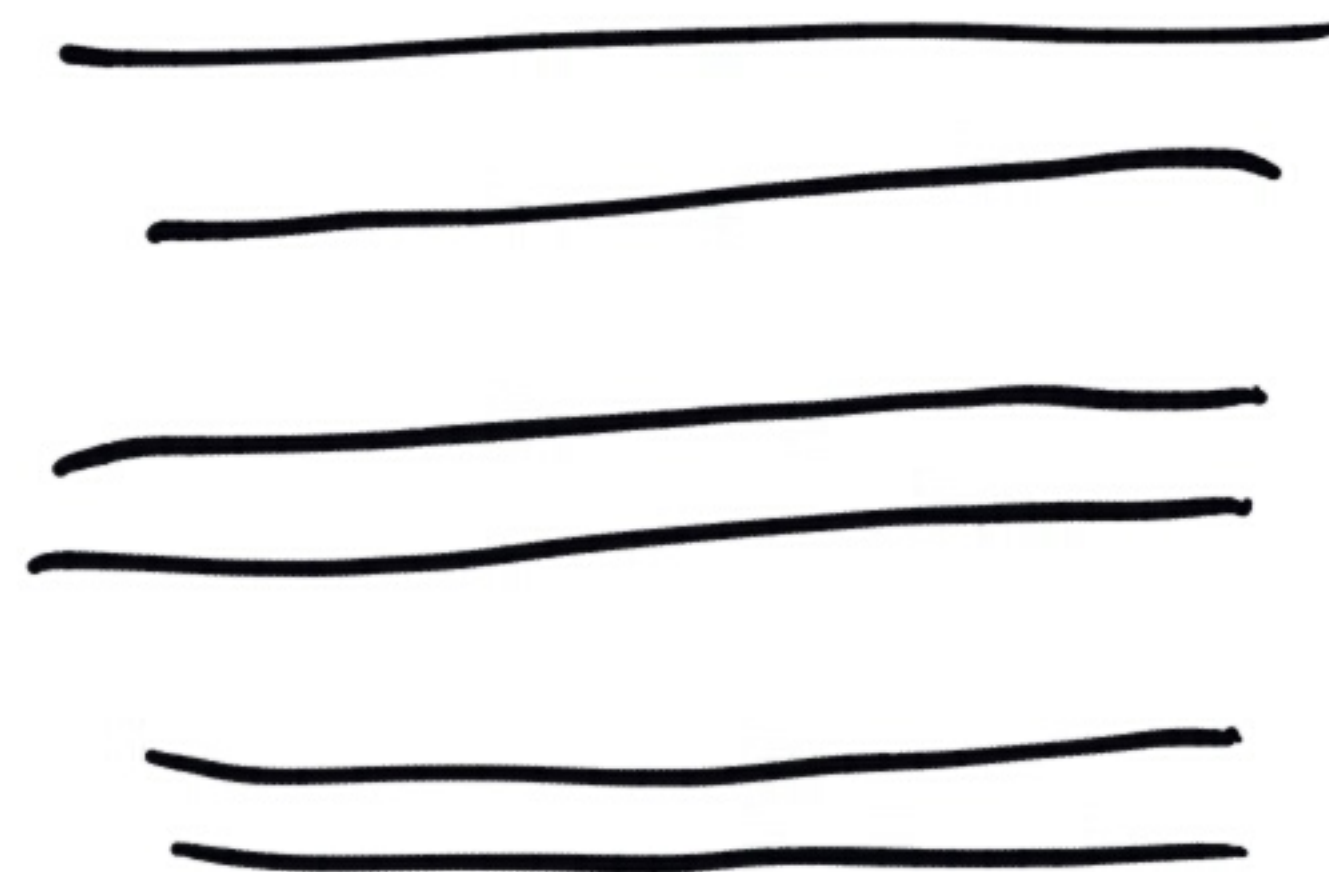
1 Round Robin Scheduling in

Processor

requests



priority
queues



[2] I/O Processing e.g. printer

Queue Implementation

Using

array →

Linked list →

Circular array →

Array-based Queue

int Max-size = 10

int Queue[Max-size]

Initially, front = rear = -1

When empty $\Rightarrow$

front = rear = -1

When full $\Rightarrow$

rear = Max-size - 1

Enqueue (X) $O(1)$

{ if (rear = ^{// full} MaxSize - 1) $\Rightarrow$ error

else if (rear = front = -1) ^{1st element} front = rear = 0

else { rear ++
queue[rear] = X

}

Dequeue ()

$O(1)$

{ if (rear = front = -1) $\Rightarrow$ error

// last element to delete

else if (front = rear)

front = rear = -1

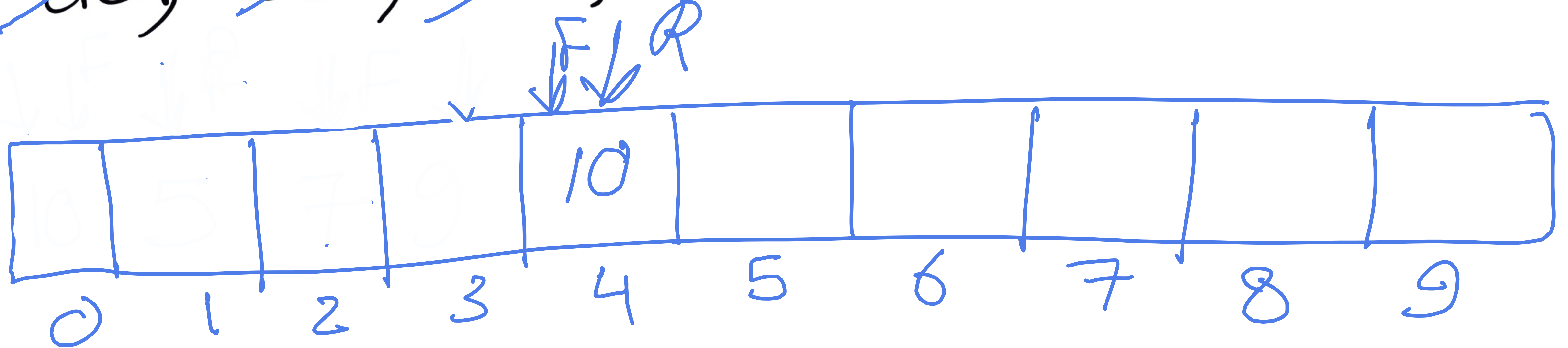
else { front ++
}

}

Trace

Assume queue [10]

~~en(10)~~, ~~en(5)~~, ~~en(7)~~, ~~en(9)~~, ~~en(10)~~,
~~de~~, ~~de~~, ~~de~~, ~~de~~

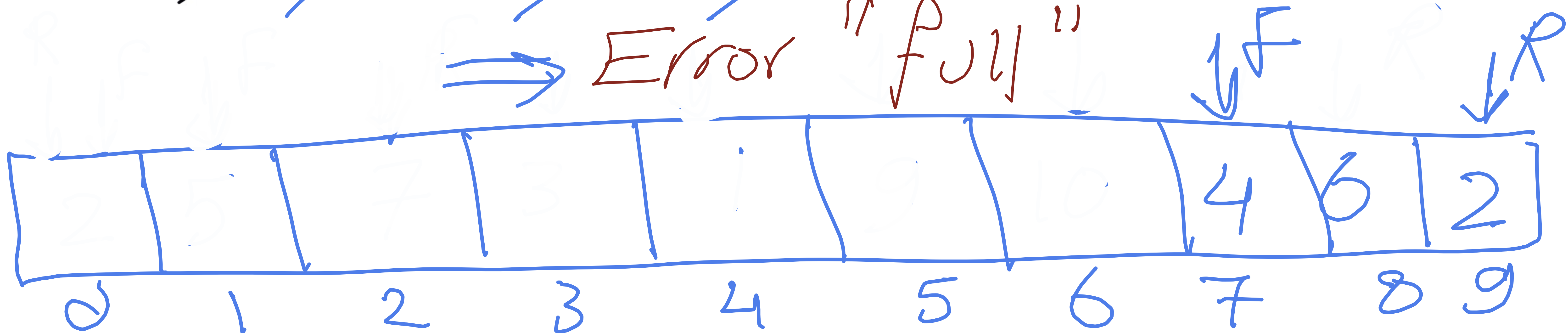


→ F
→ R

Trace : Assume queue [10]

~~en(2), en(5), en(7), de() $\Rightarrow$ 2, en(3)~~
~~en(1), en(9), en(10), en(4), en(6), de(),~~
~~en(2), de(), de(), de(), de(), en(1)~~

$\Rightarrow$ Error "full"



What is the Problem ?

(array-based queue)

When dequeue cells are not used
queue may be full while
it contains empty places

Can we do something?

Circular array

Circular Queue

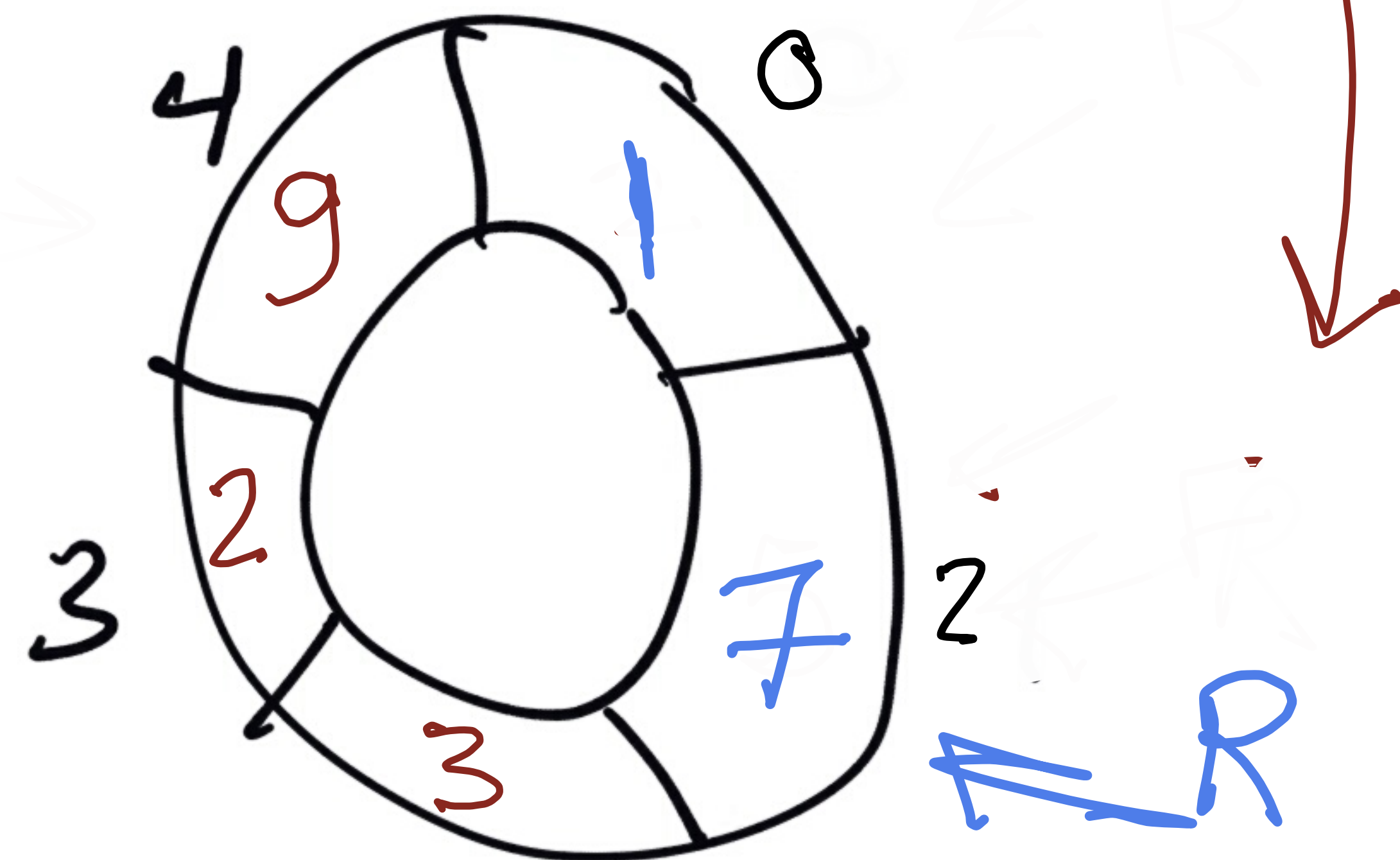
-1 $\rightarrow$ F
 $\rightarrow$ R

Max_Size = 5

Initially, front = rear =

int Queue[Max_Size]

en(1), en(7)



en(2) ✓

en(5) ✓

en(3) ✓

de() ✓

en(2), en(9), de()

enqueue (X)

{ if (front = (rear + 1) % Max-Size)

// full

Print "error"

$O(1)$

else if (front = rear = -1)

// empty

$\Rightarrow$

front = rear = 0

else { rear = (rear + 1) % Max-Size

queue[rear] = X

}

dequeue ()

$O(1)$

{ if (front = rear = -1) $\Rightarrow$ error
// only one element

else if (front = rear) $\Rightarrow$ front = rear = -1

else { return queue[front]
front = (front + 1) % Max_Size

}

Circular array-based queue

is full when ----

$$\text{rear} = \text{front} - 1$$

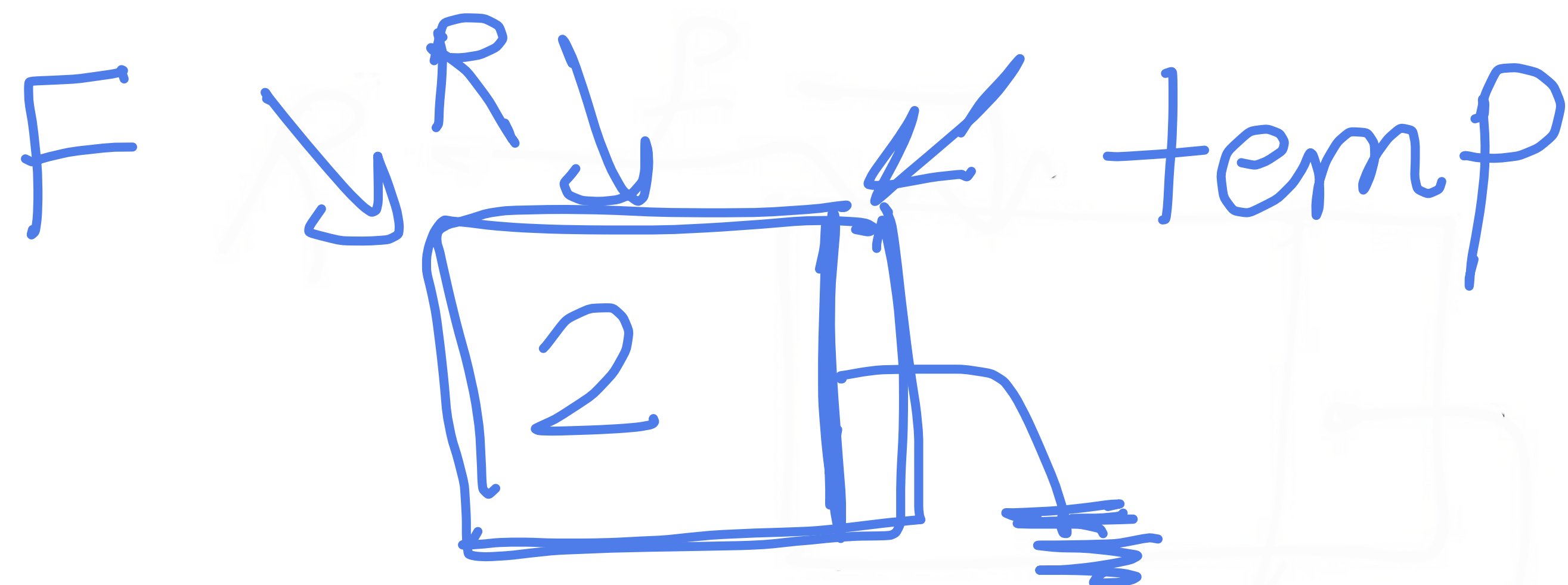
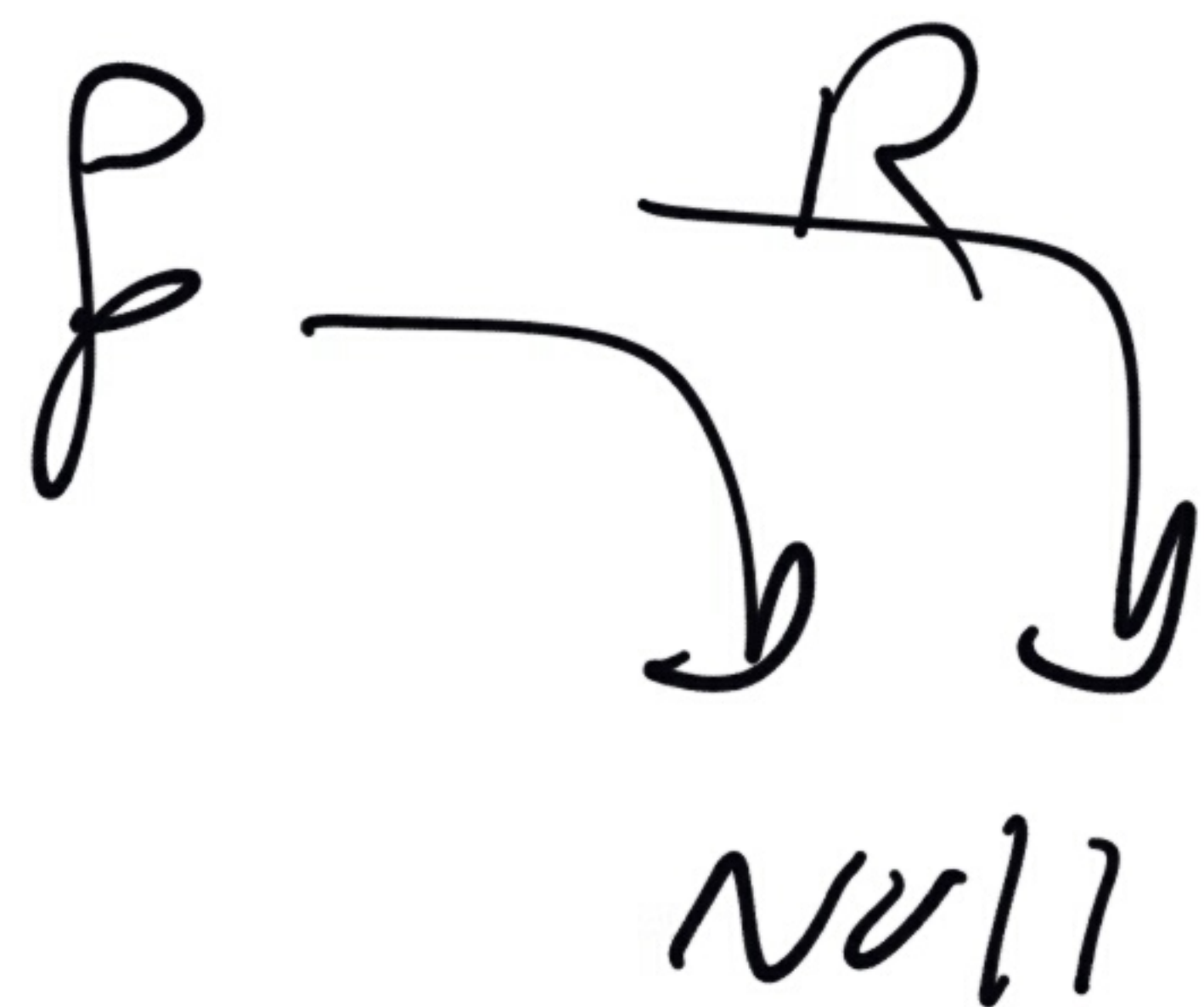
$$\text{rear} = \text{front} + 1$$

$$\text{front} = \text{rear} + 1$$

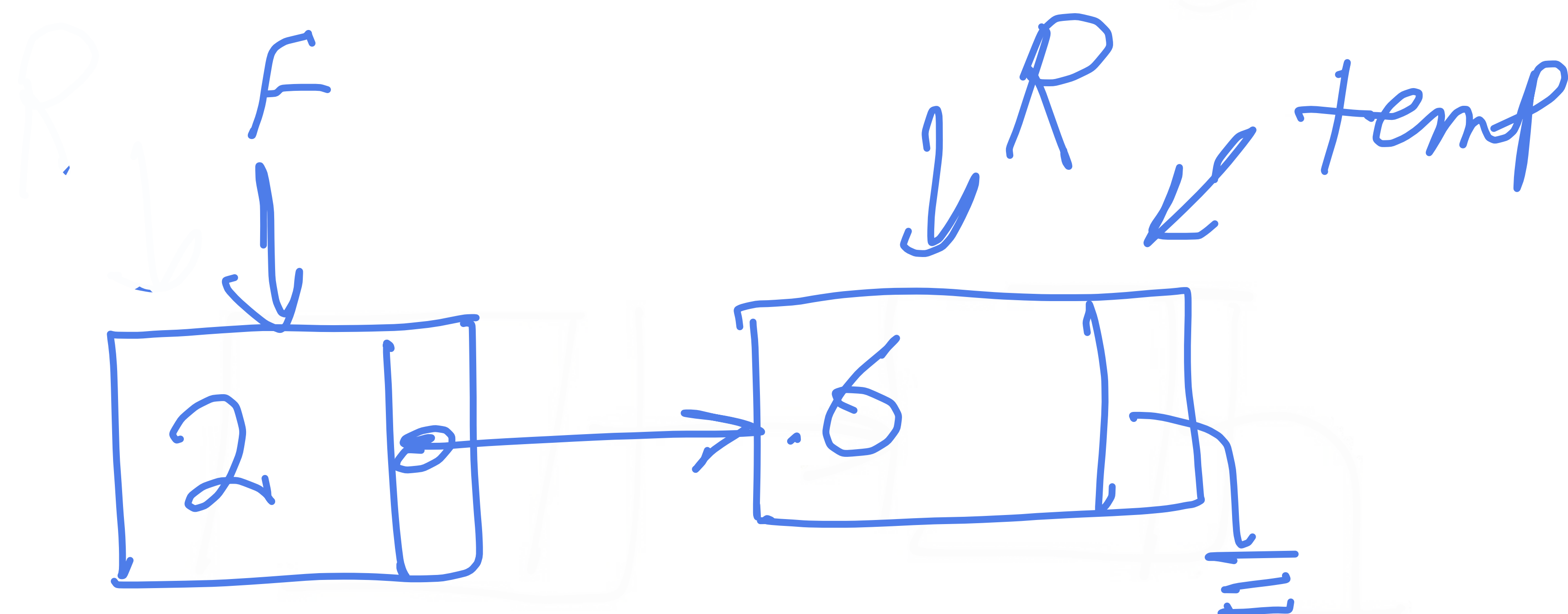
Linked list - based Queue

Initially $\text{front} = \text{rear} = \text{Null}$

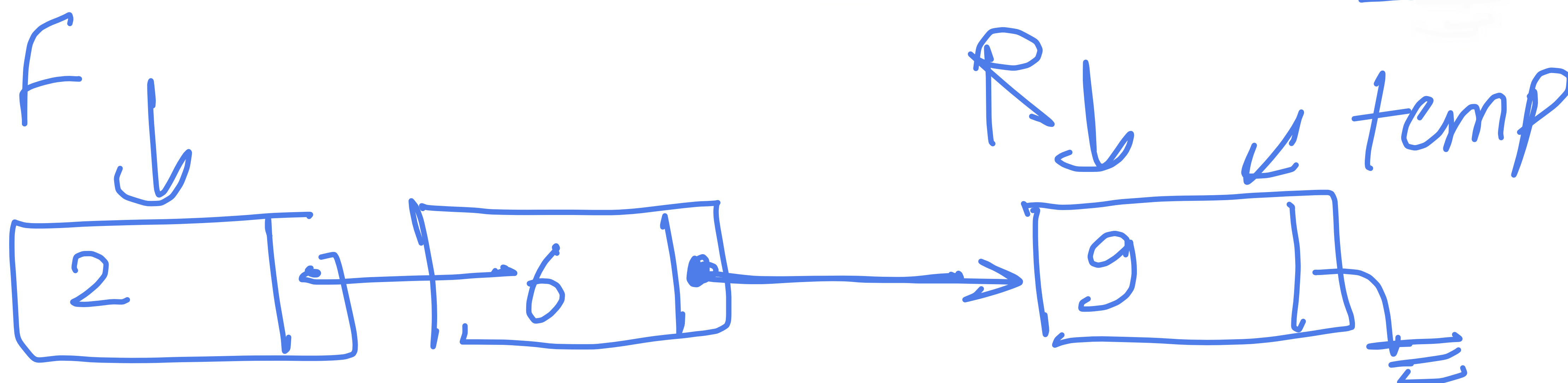
enqueue from tail/rear (insert at end)
dequeue from front (delete at top)



en(2)



en(6)



en(9)

enqueue (x)

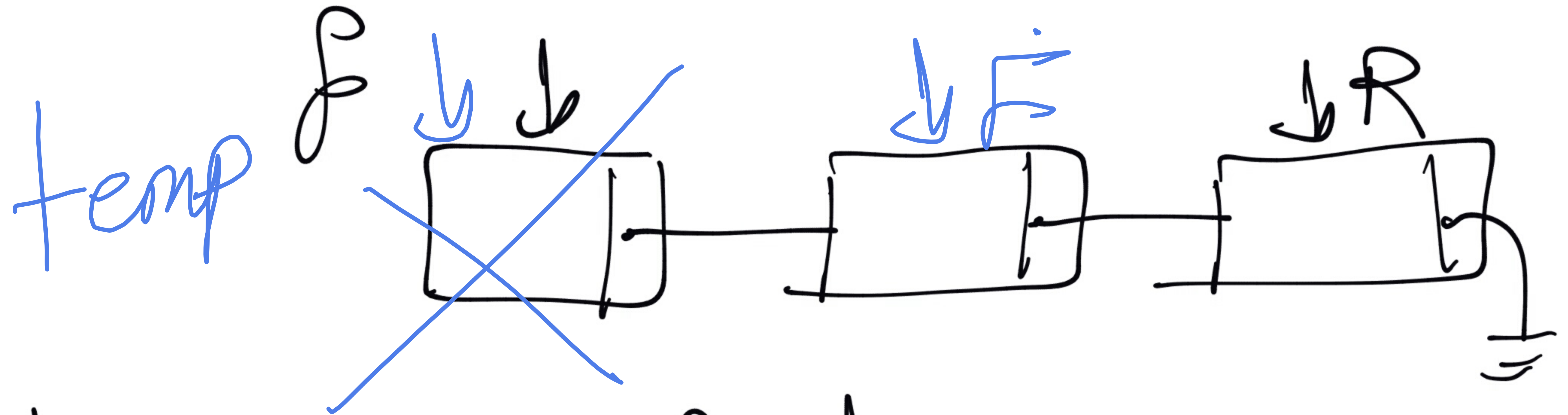
$O(1)$

```
{ Create node = temp  
  temp.data = x  
  temp->next = null
```

If (^{// empty} front = rear = null) $\Rightarrow$ front = rear = temp

```
else { rear->next = temp  
      }  
      rear = temp
```

```
}
```

dequeue from front

Dequeue()

{ temp = front
// empty

If (rear = front = null) $\Rightarrow$ error

else if (front = rear) $\Rightarrow$ front = rear = null
// last element

else { front = temp $\rightarrow$ next
free(temp)
}

}

Trace

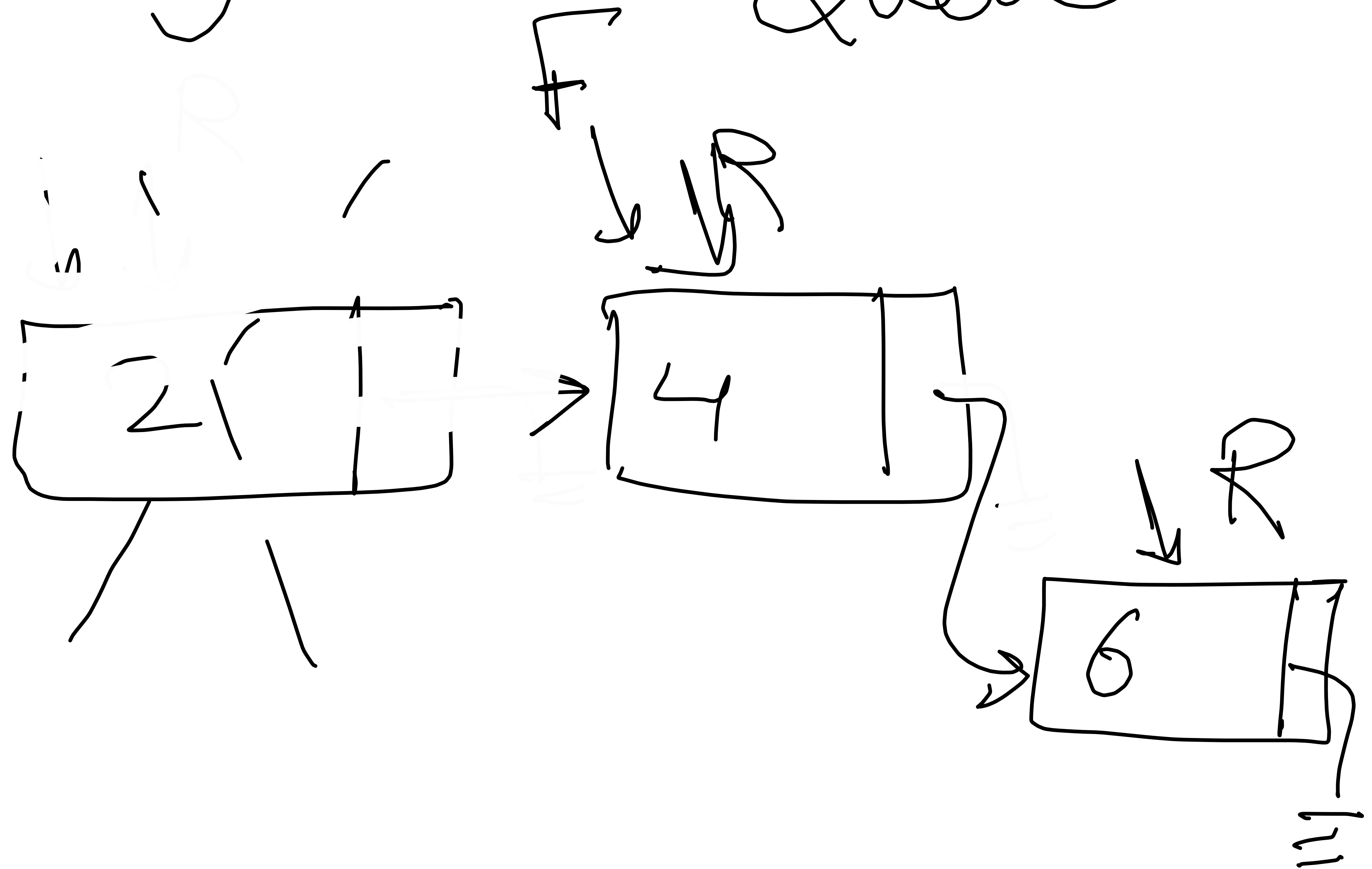
en(2) ✓

en(4) ✓

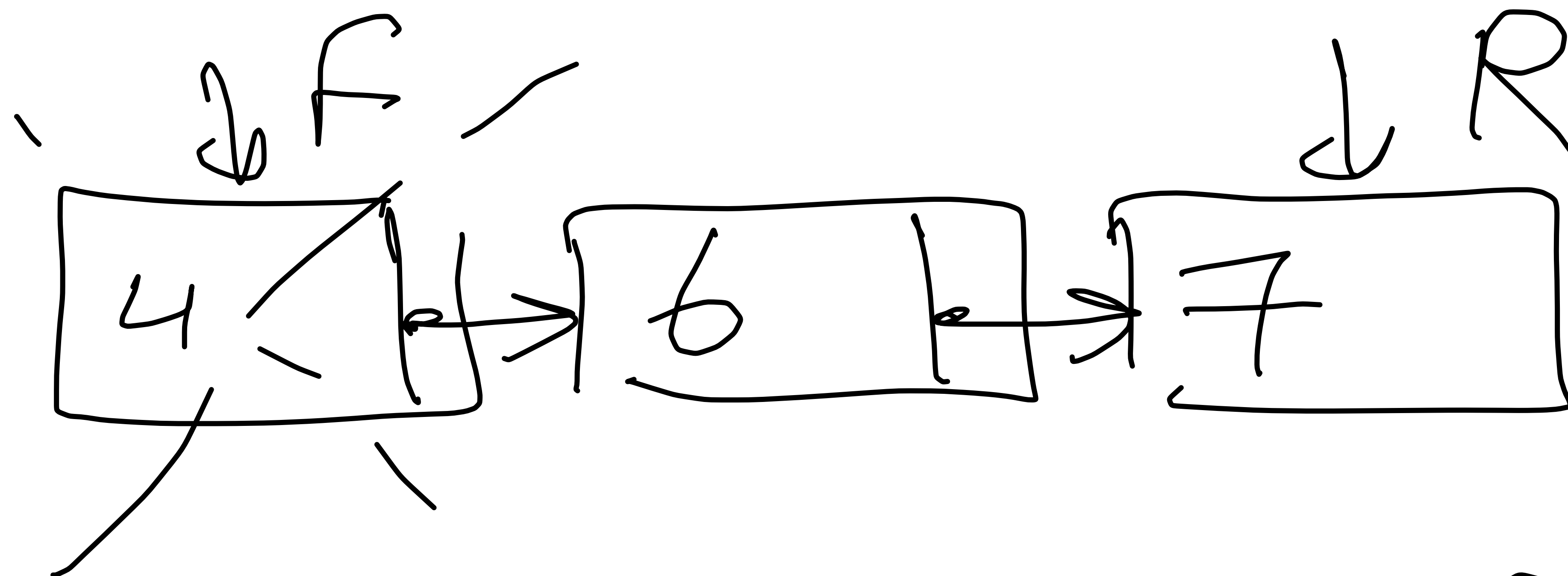
en(6) ✓

de() ⇒ ✓

Using linked-list-based Queue



en (7)



dec)

